HTML & CSS PRACTICAL ASSIGNMENT

45 Tasks | 9 Modules

Submitted by: _____

Course / Class: BCA AI – First Year

Roll Number: _____

Submitted to: _____

College: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my teacher for giving me this practical assignment on HTML and CSS. It helped me understand the structure of web pages, forms, tables, links, images, layouts and responsive design. I also learned how to use development tools and organize web projects.

Index

Module 1: Setup & Git — Tasks 1–5

Module 2: HTML Basics — Tasks 6–10

Module 3: Links, Images & Lists — Tasks 11–15

Module 4: Forms, Tables & Metadata — Tasks 16–20

Module 5: CSS Fundamentals — Tasks 21–25

Module 6: Box Model & Positioning — Tasks 26–30

Module 7: Flexbox — Tasks 31–35

Module 8: CSS Grid — Tasks 36–40

Module 9: Responsive Design & Integration — Tasks 41–45

Module 1: Setup & Git

Task 1: What happens when we type a URL?

When we type a URL in a browser, the browser checks the address and finds the website's server using DNS. It sends an HTTP or HTTPS request to the server. The server returns files such as HTML, CSS, JavaScript and images. The browser reads these files, builds the page and displays it on the screen.

Task 2: Install VS Code, Node.js and Live Server

Install Visual Studio Code from its official website and install Node.js from its official website. In VS Code, open Extensions and install Live Server. Verify the installations and take a screenshot showing VS Code, the Node.js version in the terminal, and the Live Server extension.

```
node -v
npm -v
```

Practical note: Attach your own screenshot here. Do not use a sample screenshot as proof.

Task 3: Terminal navigation practice

A terminal lets us navigate folders and run commands. Practise the following commands in the terminal.

Task 3: Terminal navigation practice (commands)

Use these commands according to your operating system.

```
pwd          # show current folder (macOS/Linux)
cd folder    # enter a folder
cd ..        # go to parent folder
ls           # list files (macOS/Linux)
dir          # list files (Windows)
mkdir html-project
cd html-project
```

Task 4: Create a Git repository and make commits

Git tracks changes in files. Create a project folder, initialize Git, add a file and commit changes. A meaningful commit message explains what changed.

```
git init
git add index.html
git commit -m "Add initial HTML page"
git status
```

Task 5: Push the project to GitHub

Create a new repository on GitHub. Follow the repository page's instructions to connect your local folder and push the files. Copy the repository URL and include it in your submitted assignment.

```
git branch -M main
git remote add origin YOUR_REPOSITORY_URL
git push -u origin main
```

Practical note: Replace YOUR_REPOSITORY_URL with your own repository URL and attach a screenshot of the repository.

Module 2: HTML Basics

Task 6: Create a basic HTML page

HTML provides the structure of a web page. The title appears on the browser tab, while headings appear in the page.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
  <h2>About Me</h2>
  <p>This is my first web page.</p>
</body>
</html>
```

Task 7: Text-only resume page

A resume page can be created using headings, paragraphs and lists without images or CSS.

```
<h1>Shalini Gangwar</h1>
<p>Email: your-email@example.com</p>
<h2>Career Objective</h2>
<p>To learn and grow as a software developer.</p>
<h2>Education</h2>
<p>BCA AI — Invertis University</p>
<h2>Skills</h2>
<ul><li>HTML</li><li>CSS</li><li>Java</li></ul>
```

Task 8: Use semantic HTML containers

Semantic tags describe the purpose of each section and make the page easier to understand.

```
<header><h1>My Resume</h1></header>
<nav><a href="#education">Education</a></nav>
<main>
  <section><h2>Objective</h2><p>To build web skills.</p></section>
  <section id="education"><h2>Education</h2><p>BCA AI</p></section>
</main>
<footer><p>Contact: your-email@example.com</p></footer>
```

Task 9: Debug a buggy HTML file

Common HTML mistakes include missing closing tags, misspelled attributes, missing quotation marks and incorrect nesting. This corrected example uses valid structure.

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Correct Page</title></head>
<body>
  <h1>My Page</h1>
  <p>This paragraph is correctly closed.</p>
  <a href="https://example.com">Visit Example</a>
</body>
</html>
```

Task 10: Personal introduction page

The finished page introduces the student and can be opened in a browser using Live Server.

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><title>About Me</title></head>
<body><h1>About Me</h1><p>Hello! I am a BCA AI student.</p>
```

```
<h2>My Interests</h2><p>I enjoy learning technology and programming.</p>
<h2>My Goal</h2><p>I want to become a skilled developer.</p></body></html>
```

Module 3: Links, Images & Lists

Task 11: Portfolio with internal navigation

Internal links take the visitor to a section on the same page. The href value must match the target section's id.

```
<nav><a href="#about">About</a> | <a href="#projects">Projects</a> | <a href="#contact">Contact</
a></nav>
<section id="about"><h2>About</h2><p>I am learning web development.</p></section>
<section id="projects"><h2>Projects</h2><p>My first HTML page.</p></section>
<section id="contact"><h2>Contact</h2><p>your-email@example.com</p></section>
```

Task 12: Add meaningful alt attributes

The alt attribute describes an image for screen readers and when the image cannot load. Use a useful description instead of leaving alt empty for meaningful images.

```
<img src="college-building.jpg" alt="Main building of my college">
<img src="laptop.jpg" alt="Laptop displaying a code editor">
```

Practical note: Use image files that actually exist in your project folder.

Task 13: My college / course information using lists

Lists organize information clearly. Use ul for unordered items and ol when order matters.

```
<h1>My College and Course</h1>
<ul><li>College: Invertis University</li><li>Course: BCA AI</li><li>Year: First Year</li></ul>
<h2>Subjects to Learn</h2>
<ol><li>HTML</li><li>CSS</li><li>Programming Fundamentals</li></ol>
```

Task 14: Improve a page with semantic tags

Use header, nav, main, section, article and footer according to their meaning. Avoid using tags only for their default appearance.

```
<header><h1>Student Blog</h1></header>
<nav><a href="#post">Latest Post</a></nav>
<main><article id="post"><h2>Learning HTML</h2><p>HTML gives a page its structure.</p></article></
main>
<footer><p>Student Blog</p></footer>
```

Task 15: Personal blog / portfolio page

This combines a heading, navigation, sections, a project list and contact details.

```
<header><h1>My Portfolio</h1><p>BCA AI Student</p></header>
<nav><a href="#skills">Skills</a> | <a href="#work">Work</a> | <a href="#contact">Contact</a></nav>
<main><section id="skills"><h2>Skills</h2><ul><li>HTML</li><li>CSS</li><li>Java</li></ul></section>
<section id="work"><h2>Projects</h2><p>Personal introduction page</p></section>
<section id="contact"><h2>Contact</h2><p>your-email@example.com</p></section></main>
<footer><p>Thank you for visiting.</p></footer>
```

Module 4: Forms, Tables & Metadata

Task 16: Student registration form

Forms collect information from users. Labels should be connected to inputs using matching for and id values.

```
<form>
<label for="name">Full Name:</label><input id="name" name="name" type="text" required><br>
<label for="email">Email:</label><input id="email" name="email" type="email" required><br>
<label for="dob">Date of Birth:</label><input id="dob" name="dob" type="date"><br>
<label for="course">Course:</label><select id="course" name="course"><option>BCA AI</
option><option>BCA</option></select><br>
<label><input type="checkbox" name="agree" required> I confirm the details.</label><br>
<button type="submit">Register</button>
</form>
```

Task 17: Job application form with input types

Different input types help collect the correct kind of data. This example includes text, email, telephone, date, number, file, radio, checkbox and textarea.

```
<form>
<label>Name <input type="text" name="name" required></label><br>
<label>Email <input type="email" name="email" required></label><br>
<label>Phone <input type="tel" name="phone"></label><br>
<label>Available from <input type="date" name="start"></label><br>
<label>Experience (years) <input type="number" name="experience" min="0"></label><br>
<label>Resume <input type="file" name="resume"></label><br>
<p>Work type: <label><input type="radio" name="type" value="onsite"> On-site</label> <label><input
type="radio" name="type" value="remote"> Remote</label></p>
<label><input type="checkbox" name="consent"> I agree to be contacted.</label><br>
```

```
<label>Cover note <textarea name="cover-note"></textarea></label><br><button>Submit Application</
button>
</form>
```

Task 18: Product size chart table

Tables display related information in rows and columns. Use th for headings and caption to describe the table.

```
<table border="1">
<caption>T-Shirt Size Chart</caption>
<tr><th>Size</th><th>Chest (inches)</th><th>Length (inches)</th></tr>
<tr><td>S</td><td>34–36</td><td>26</td></tr>
<tr><td>M</td><td>38–40</td><td>27</td></tr>
<tr><td>L</td><td>42–44</td><td>28</td></tr>
</table>
```

Task 19: Add metadata to HTML files

Metadata gives the browser information about the page. Add charset, viewport, title and description in the head element.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="A student portfolio website">
  <title>Student Portfolio</title>
</head>
```

Task 20: Multi-page HTML project

Create four files named index.html, about.html, registration.html and contact.html. Add the same navigation to each page and link each file with a relative path.

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="registration.html">Registration</a>
  <a href="contact.html">Contact</a>
</nav>
<!-- Put the navigation inside the body of every page. -->
```

Module 5: CSS Fundamentals

Task 21: Style an HTML-only page

CSS controls colours, spacing, fonts and layout. Save the following in style.css and link it from the HTML head using <link rel="stylesheet" href="style.css">.

```
body { font-family: Arial, sans-serif; margin: 24px; background: #f4f7fb; color: #243047; }
h1 { color: #173b7a; }
p { line-height: 1.6; }
```

Task 22: Use four selector types

CSS selectors target elements to style. This example uses an element selector, class selector, ID selector and descendant selector.

```
p { line-height: 1.5; }       /* element selector */
.card { padding: 16px; }       /* class selector */
#main-title { color: navy; }   /* ID selector */
nav a { text-decoration: none; } /* descendant selector */
```

Task 23: Resume page with font pairing and colour palette

Use a readable body font and a clear heading font. Limit the palette to a few consistent colours.

```
body { font-family: Arial, sans-serif; color: #263238; background: #f5f7fa; }
h1, h2 { font-family: Georgia, serif; color: #173b7a; }
.resume { max-width: 760px; margin: 24px auto; padding: 24px; background: white; }
a { color: #087e8b; }
```

Task 24: Design tokens using CSS variables

CSS custom properties store reusable design values. Define them inside :root and use var() to apply them.

```
:root { --primary: #173b7a; --accent: #087e8b; --space: 16px; --text-size: 16px; }
body { color: #263238; font-size: var(--text-size); }
h1 { color: var(--primary); }
.card { padding: var(--space); border-top: 4px solid var(--accent); }
```

Task 25: Styled multi-page project and design note

Link one shared CSS file to all project pages so the design remains consistent. Design note: I used a limited colour palette, readable fonts, consistent spacing and clear headings to improve usability.

```
/* style.css */
body { margin: 0; font-family: Arial, sans-serif; line-height: 1.6; color: #263238; }
```

```
header, footer { background: #173b7a; color: white; padding: 16px; }
main { max-width: 1000px; margin: auto; padding: 20px; }
a { color: #087e8b; }
```

Module 6: Box Model & Positioning

Task 26: Explain the CSS box model

Every HTML element is represented as a box made of content, padding, border and margin. Content is the text or image; padding is space inside the border; border surrounds the padding and content; margin is space outside the border.

```
/* Example annotation */
.box { width: 200px; padding: 20px; border: 5px solid navy; margin: 15px; box-sizing: border-box; }
```

Practical note: Draw a labelled rectangle in your notebook showing Content → Padding → Border → Margin.

Task 27: Three card styles

Border radius rounds corners and background colours distinguish card styles.

```
.card { padding: 18px; margin: 10px; border-radius: 12px; }
.card-one { background: #eef4ff; border: 1px solid #b7c9ee; }
.card-two { background: #e8f7f2; border-left: 5px solid #17866b; }
.card-three { background: #fff3e5; border: 2px dashed #d78b2f; }
```

Task 28: Fix overflow issues

Overflow often occurs when an element is wider than its container or when fixed widths are used. Use max-width, box-sizing and responsive images.

```
* { box-sizing: border-box; }
.container { width: 100%; max-width: 900px; margin: auto; padding: 16px; }
img { max-width: 100%; height: auto; }
.content { overflow-wrap: anywhere; }
```

Task 29: Landing-page hero using position and z-index

A hero section is the main introductory section of a page. Positioning and z-index control how elements overlap.

```
.hero { position: relative; min-height: 280px; padding: 40px; background: #eaf0ff; overflow: hidden;
 }
.hero-badge { position: absolute; top: 16px; right: 16px; z-index: 2; background: navy; color:
white; padding: 8px 12px; border-radius: 20px; }
.hero-content { position: relative; z-index: 1; }
```

Task 30: Profile card / product card

A card groups related information into one component. Combine a title, short description and link.

```
<div class="profile-card"><h2>Student Profile</h2><p>BCA AI student learning web development.</p><a
href="#contact">Contact</a></div>

.profile-card { max-width: 320px; padding: 20px; border-radius: 14px; background: #f4f7fb; border:
1px solid #d8dfeb; box-shadow: 0 4px 12px #00000014; }
```

Module 7: Flexbox

Task 31: Responsive menu using Flexbox

Flexbox arranges items in a row or column and can distribute free space.

```
<nav class="menu"><a href="#home">Home</a><a href="#about">About</a><a href="#contact">Contact</a></
nav>
.menu { display: flex; gap: 20px; justify-content: center; flex-wrap: wrap; padding: 16px;
background: #eaf0ff; }
.menu a { text-decoration: none; }
```

Task 32: Centered login form using Flexbox

Use a flex container to center the login card both vertically and horizontally.

```
<main class="login-wrap"><form class="login"><h1>Login</h1><label>Email <input type="email"></
label><label>Password <input type="password"></label><button>Login</button></form></main>
.login-wrap { min-height: 100vh; display: flex; justify-content: center; align-items: center;
padding: 16px; }
.login { display: flex; flex-direction: column; gap: 12px; width: min(100%, 320px); padding: 24px; }
```

Task 33: Product listing using flex-wrap

The flex-wrap property moves cards to another line when there is not enough space.

```
<div class="products"><article class="product">Product 1</article><article class="product">Product
2</article><article class="product">Product 3</article></div>
.products { display: flex; flex-wrap: wrap; gap: 16px; }
.product { flex: 1 1 180px; padding: 20px; border: 1px solid #ccd4e0; border-radius: 10px; }
```

Task 34: Responsive navbar and hero using Flexbox

The navigation uses flex-wrap so it can fit on small screens. The hero aligns text and a call-to-action.

```
<header class="navbar"><strong>MySite</strong><nav><a href="#features">Features</a> <a
href="#contact">Contact</a></nav></header>
<section class="hero"><div><h1>Learn Web Development</h1><p>Build your first website.</p><a
href="#features">Get Started</a></div></section>
.navbar { display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap:
12px; padding: 16px; }
.hero { display: flex; align-items: center; min-height: 260px; padding: 32px; background: #eef4ff; }
```

Task 35: Finished Flexbox landing-page section

Combine a navbar, hero and three feature cards. Flexbox makes the cards wrap on smaller screens.

```
.features { display: flex; flex-wrap: wrap; gap: 16px; }
.feature { flex: 1 1 220px; padding: 18px; border-radius: 12px; background: #f4f7fb; }
@media (max-width: 600px) { .navbar { align-items: flex-start; flex-direction: column; } }
```

Module 8: CSS Grid

Task 36: Photo gallery using CSS Grid

CSS Grid arranges items in rows and columns. Use auto-fit and minmax to adapt the number of columns.

```
<div class="gallery"><div>Photo 1</div><div>Photo 2</div><div>Photo 3</div><div>Photo 4</div></div>
.gallery { display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 12px; }
.gallery div { min-height: 100px; display: grid; place-items: center; background: #eaf0ff; border-
radius: 8px; }
```

Task 37: Responsive product grid

The minmax function sets a minimum and flexible maximum width for each card.

```
.product-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap:
16px; }
.product-card { padding: 16px; border: 1px solid #d8dfeb; border-radius: 12px; }
.product-card img { width: 100%; height: auto; }
```

Task 38: Dashboard with named grid areas

Named grid areas make complex layouts easier to understand and maintain.

```
<div class="dashboard"><header>Header</header><aside>Sidebar</aside><main>Main content</
main><footer>Footer</footer></div>
.dashboard { display: grid; gap: 10px; grid-template-areas: "header header" "sidebar main" "footer
footer"; grid-template-columns: 180px 1fr; }
.dashboard header { grid-area: header; } .dashboard aside { grid-area: sidebar; }
.dashboard main { grid-area: main; } .dashboard footer { grid-area: footer; }
.dashboard > * { padding: 16px; background: #eef4ff; }
@media (max-width: 600px) { .dashboard { grid-template-areas: "header" "sidebar" "main" "footer";
grid-template-columns: 1fr; } }
```

Task 39: Responsive product grid with header and sidebar

Grid can combine a header, sidebar and product list. On narrow screens, the layout changes to one column.

```
<div class="shop"><header>Online Store</header><aside>Categories</aside><main
class="items"><article>Item 1</article><article>Item 2</article><article>Item 3</article></main></
div>
.shop { display: grid; grid-template-columns: 180px 1fr; gap: 16px; }
.shop header { grid-column: 1 / -1; padding: 16px; background: #173b7a; color: white; }
.items { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 12px; }
.items article, .shop aside { padding: 16px; background: #eef4ff; }
@media (max-width: 600px) { .shop { grid-template-columns: 1fr; } .shop header { grid-column: auto;
} }
```

Task 40: Finished Grid-based project

Create a page with a header, gallery or product grid, and footer. Check that the layout stays usable on a phone.

```
.page { display: grid; grid-template-columns: repeat(12, 1fr); gap: 16px; padding: 16px; }
.page-header, .page-footer { grid-column: 1 / -1; padding: 18px; background: #173b7a; color: white;
}
.page-main { grid-column: 1 / -1; display: grid; grid-template-columns: repeat(auto-fit, minmax(
180px, 1fr)); gap: 16px; }
@media (max-width: 600px) { .page { gap: 10px; padding: 10px; } }
```

Module 9: Responsive Design & Integration

Task 41: Add two breakpoints

Media queries apply CSS only when the viewport matches a condition. These two breakpoints adjust spacing and columns.

```
@media (max-width: 768px) { .layout { grid-template-columns: 1fr 1fr; } }
```

```
@media (max-width: 480px) { .layout { grid-template-columns: 1fr; } .container { padding: 12px; } }
```

Task 42: Audit responsiveness

Test the page at desktop, tablet and mobile widths using the browser's responsive design tools. Look for horizontal scrolling, tiny text, clipped content and overlapping elements. Fix them with flexible widths, wrapping, responsive images and media queries.

```
*, *::before, *::after { box-sizing: border-box; }
img, video { max-width: 100%; height: auto; }
.container { width: min(100% - 32px, 1100px); margin-inline: auto; }
body { overflow-wrap: anywhere; }
```

Task 43: Reference landing page using Grid and Flexbox

Use Grid for the main page structure and Flexbox for navigation or button alignment.

```
<header class="topbar"><strong>LearnWeb</strong><nav><a href="#about">About</a><a
href="#courses">Courses</a></nav></header>
<main class="landing"><section><h1>Build your future online</h1><p>Learn HTML and CSS step by step.
</p><a href="#courses">Explore Courses</a></section><aside>Start learning today</aside></main>
.topbar { display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap:
12px; padding: 16px; }
.topbar nav { display: flex; gap: 16px; flex-wrap: wrap; }
.landing { display: grid; grid-template-columns: 2fr 1fr; gap: 20px; padding: 24px; }
.landing section, .landing aside { padding: 24px; background: #eef4ff; border-radius: 12px; }
@media (max-width: 650px) { .landing { grid-template-columns: 1fr; } }
```

Task 44: Push the landing-page skeleton to GitHub

Create the landing page files, test them locally and push the project to GitHub. Use meaningful commit messages and include the repository link in your submission.

```
git add index.html style.css
git commit -m "Create responsive landing page"
git push
```

Practical note: Write your actual GitHub repository URL here: _____

Task 45: Submit the finished responsive landing page

The final project should contain index.html and style.css (plus any image files used). Confirm that navigation links work, images have alt text, forms have labels, and the page works at desktop and mobile widths. Include screenshots and the GitHub link.

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,
initial-scale=1.0"><title>LearnWeb</title><link rel="stylesheet" href="style.css"></head>
<body><header class="topbar"><strong>LearnWeb</strong><nav><a href="#about">About</a><a
href="#courses">Courses</a></nav></header>
<main><section class="hero" id="about"><h1>Learn HTML & CSS</h1><p>Create your first responsive
website.</p><a class="btn" href="#courses">Get Started</a></section>
<section id="courses"><h2>What You Will Learn</h2><div class="cards"><article>HTML Structure</
article><article>CSS Styling</article><article>Responsive Design</article></div></section></main>
<footer><p>Student Web Project</p></footer></body></html>

/* style.css */
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; line-height: 1.6; color: #263238; }
.topbar { display: flex; justify-content: space-between; flex-wrap: wrap; gap: 12px; padding: 16px
5%; background: #173b7a; color: white; }
.topbar nav { display: flex; gap: 16px; }
.topbar a, .btn { color: white; }
.hero { padding: 56px 5%; background: #eef4ff; }
.btn { display: inline-block; padding: 10px 16px; background: #087e8b; border-radius: 6px; text-
decoration: none; }
#courses { padding: 24px 5%; }
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 16px; }
.cards article { padding: 20px; border-radius: 10px; background: #f4f7fb; }
footer { padding: 16px 5%; background: #173b7a; color: white; }
@media (max-width: 480px) { .hero { padding: 32px 5%; } }
```

Conclusion

Through these 45 practical tasks, I learned the fundamentals of HTML and CSS, semantic page structure, links, images, lists, forms, tables, metadata, CSS selectors, variables, the box model, positioning, Flexbox, Grid and responsive design. I also practised terminal commands, Git and GitHub. These skills provide a foundation for building accessible and responsive websites.

Submission Checklist

- ■ Cover page and index completed.
- ■ All code examples written neatly and tested where possible.
- ■ Screenshots added for installation and project proof tasks.

■ Actual GitHub repository links added for Tasks 5 and 44–45.

■ HTML files and CSS files are saved with correct file extensions.

■ Final project checked on desktop and mobile screen sizes.